

Festival

خيرته في مهرجان تلعب، والجائزة الكبرى هي رحلة إلى لاجونا كولورادو. تستخدم الدنانير التي لديها لشراء القسائم. قد يؤدي شراء قسيمة إلى الحصول على دنانير إضافية. الهدف هو الحصول على أكبر عدد ممكن من القسائم.

تبدأ اللعبة بـ A ديناراً، والتي يمكنها استخدامها لشراء قسائم. يوجد N قسيمة، مرقمة من 0 إلى $N - 1$. تبلغ تكلفة القسيمة i ، $P[i]$ ديناراً ($0 \leq i < N$). وبالتالي، يتعين على خيرته أن تدفع $P[i]$ ديناراً من أجل شرائها (ويجب أن يكون لديها ما لا يقل عن $P[i]$ ديناراً قبل الشراء). يمكنها شراء كل قسيمة مرة واحدة على الأكثر.

بالإضافة إلى ذلك، لكل قسيمة i ($0 \leq i < N$) يتم تعيين نوع، يُشار إليه بـ $T[i]$ ، وهو عدد صحيح يتراوح بين 1 و 4، محتواه. بعد أن تشتري خيرته القسيمة i ، يتم ضرب عدد الدينارات المتبقية لديها في $T[i]$. أي إذا كان لديها X ديناراً في مرحلة ما من اللعبة، واشترت القسيمة i (وهو ما يتطلب $X \geq P[i]$)، فسيكون لديها $(X - P[i]) \cdot T[i]$ ديناراً بعد الشراء.

مهمتك هي تحديد القسائم التي يجب أن تشتريها خيرته وبأي ترتيب، لتعظيم إجمالي عدد القسائم التي لديها في النهاية. إذا كان هناك أكثر من تسلسل واحد من المشتريات يحقق ذلك، يمكنك الإبلاغ عن أي منها.

Implementation Details

You should implement the following procedure

```
std::vector<int> max_coupons(int A, std::vector<int> P,
                             std::vector<int> T)
```

- A : العدد الأولي للدنانير التي تمتلكها خيرته.
 - P : مصفوفة طولها N تحدد أسعار القسائم.
 - T : مصفوفة طولها N تحدد أنواع القسائم.
 - يتم استدعاء هذا الإجراء مرة واحدة فقط لكل حالة اختبار. يجب أن يُرجع الإجراء مصفوفة R ، والتي تحدد مشتريات خيرته على النحو التالي:
 - يجب أن يكون طول R هو الحد الأقصى لعدد القسائم التي يمكنها شراؤها.
 - عناصر المصفوفة هي دلائل القسائم التي يجب عليها شراؤها، بالترتيب الزمني. أي أنها تشتري الكوبون $R[0]$ أولاً، ثم الكوبون $R[1]$ ثانياً، وهكذا.
 - يجب أن تكون جميع عناصر R فريدة من نوعها.
- إذا لم يكن بالإمكان شراء أي قسائم، يجب أن يكون R مصفوفة فارغة.

Constraints

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $0 \leq i < N$ for each i such that $1 \leq P[i] \leq 10^9$
- $0 \leq i < N$ for each i such that $1 \leq T[i] \leq 4$

Subtasks

Subtask	Score	Additional Constraints
1	5	$T[i] = 1$ for each i such that $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ for each i such that $0 \leq i < N$.
3	12	$T[i] \leq 2$ for each i such that $0 \leq i < N$.
4	15	$N \leq 70$
5	27	خيرہ يمكن شراء جميع القسائم N (بترتيب ما).
6	16	$(A - P[i]) \cdot T[i] < A$ for each i such that $0 \leq i < N$.
7	18	No additional constraints.

Examples

Example 1

.Consider the following call

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

لدى خيرہ في البداية $A = 13$ ديناراً. يمكنها شراء 3 قسائم بالترتيب الموضح أدناه:

الكوبون المشتري	Coupon price	Coupon type	Number of tokens after the purchase
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

في هذا المثال، من غير الممكن أن تشتري خيرہ أكثر من 3 قسائم، وتسلسل المشتريات الموضح أعلاه هو الطريقة الوحيدة التي يمكنها من خلالها شراء 3 منها. وبالتالي، يجب أن يعود الإجراء $[2, 3, 0]$.

Example 2

.Consider the following call

```
max_coupons(9, [6, 5], [2, 3])
```

في هذا المثال يمكن لخبره شراء كلا القسيتين بأي ترتيب وبالتالي، يجب أن يعود الإجراء إما $[0, 1]$ أو $[1, 0]$.

Example 3

.Consider the following call

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

في هذا المثال، تمتلك خبره رمزاً واحداً، وهو ما لا يكفي لشراء أي كوبونات. وبالتالي، يجب أن يعود الإجراء $[]$ (مصفوفة فارغة).

Sample Grader

:Input format

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

:Output format

```
S
R[0] R[1] ... R[S-1]
```

وهنا، S هو طول المصفوفة R التي تم إرجاعها بواسطة `max_coupons`.